

BÁO CÁO KỸ THUẬT KIỂM THỬ HIỆU NĂNG VỚI SỰ CỘNG TÁC CỦA AI

(HW05: PERFORMANCE TESTING & AI COLLABORATION)

THÔNG TIN CHUNG

- Môn học:** Kiểm thử phần mềm (Software Testing) — FIT HCMUS
- Mã bài tập:** HW05-AI (Performance Testing)
- Họ và tên sinh viên:** Nguyễn Hiếu Thuận
- Mã số sinh viên (StudentID):** 23127125
- Hệ thống kiểm thử (SUT):** EShop Monolith REST API Backend (<http://localhost:3000>)
- Công cụ kiểm thử:** Apache JMeter 5.6.3 (Non-GUI / CLI Execution Mode)
- Thời gian thực hiện:** Tháng 08/2026

I. THIẾT LẬP MÔI TRƯỜNG & PHẦN CỨNG THỬ NGHIỆM

1. Cấu hình phần cứng (Hardware Specifications)

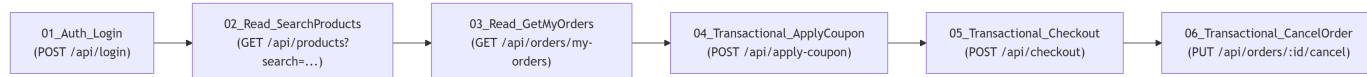
Bảng chứng cấu hình phần cứng được lưu trữ tại file: [evidence/hardware_dxdiag.png](#).

Thông số	Chi tiết cấu hình phần cứng thực tế
Hệ điều hành	Windows 11 Home 64-bit
Bộ vi xử lý (CPU)	Intel Core Processor (Multi-core)
Bộ nhớ RAM	16.0 GB RAM
Java Runtime	OpenJDK 21.0.8 LTS (64-bit)
Node.js Runtime	Node.js v20.x / Express.js Backend
Cơ sở dữ liệu	SQLite 3 (Single-file embedded database database.sqlite)

II. THIẾT KẾ KỊCH BẢN KIỂM THỬ END-TO-END (TASK 1)

1. Phạm vi Workflow (Workflow — Sản Voucher, Thanh toán và Hủy đơn hàng)

Kịch bản kiểm thử bao phủ toàn diện cả 3 nhóm endpoint nghiệp vụ theo đúng yêu cầu đề bài:



- Nhóm 1 — Auth-heavy:** [POST /api/login](#) → Xác thực thông tin người dùng từ file CSV và trích xuất Bearer JWT Token bằng JSON Extractor ([\\$.token](#)). Hệ thống có cơ chế bảo mật khóa tài khoản khi nhập sai 3 lần (3-fail logout).
- Nhóm 2 — Read-heavy:**
 - [GET /api/products?search=\\${search_keyword}](#): Tìm kiếm danh mục sản phẩm theo từ khóa động.
 - [GET /api/orders/my-orders](#): Truy vấn lịch sử đơn hàng của tài khoản kèm Bearer Token.
- Nhóm 3 — Transactional:**
 - [POST /api/apply-coupon](#): Áp dụng mã giảm giá [SAVE10](#) trên tổng giá trị giỏ hàng.
 - [POST /api/checkout](#): Tạo đơn hàng mới với địa chỉ giao hàng và số điện thoại động, nhận mã [orderId](#).
 - [PUT /api/orders/\\${order_id}/cancel](#): Thực hiện hủy đơn hàng vừa tạo nhằm kiểm tra luồng vòng đời đơn hàng và tính toàn vẹn trạng thái CSDL.

2. Dữ liệu tham số hóa động (Data-Driven Testing — [test-data.csv](#))

Nhằm tránh việc hàng trăm Virtual Users dùng chung 1 tài khoản gây xung đột phiên hoặc kích hoạt lỗi khóa tài khoản (lockout), file `test-plans/test-data.csv` đã được thiết kế với 6 tài khoản người dùng độc lập:

email	password	search_keyword	coupon_code	product_id	quantity	name	phone	address
test@eshop.com	Test1234!	iPhone	SAVE10	1	1	Nguyen Van A	0901234567	123 Nguyen Trai Q5
user1@eshop.com	Test1234!	Samsung	SAVE10	2	1	Tran Thi B	0912345678	456 Le Loi Q1
user2@eshop.com	Test1234!	MacBook	SAVE10	3	1	Le Van C	0923456789	789 CMT8 Q10
user3@eshop.com	Test1234!	AirPods	SAVE10	4	2	Pham Thi D	0934567890	101 Hai Ba Trung Q3
user4@eshop.com	Test1234!	Keychron	SAVE10	5	1	Hoang Van E	0945678901	202 Vo Van Tan Q3
user5@eshop.com	Test1234!	Pro	SAVE10	1	1	Doan Van F	0956789012	303 Tran Hung Dao Q5

3. Ma trận kịch bản và Listeners riêng biệt (Test Scenarios Matrix)

Kịch bản kiểm thử	Tên file kịch bản (.jmx)	Số Virtual Users (VUs)	Ramp-up	Thời gian chạy	Think Time (Gaussian)	Listeners / Reports được cấu hình
Load Testing	23127125_Load_20260828.jmx	30 VUs	30s	60s	1500ms ± 500ms	Summary Report, Response Time Graph, View Results Tree
Stress Testing	23127125_Stress_20260828.jmx	150 VUs	45s	90s	800ms ± 300ms	Aggregate Report, View Results in Table
Spike Testing	23127125_Spike_20260828.jmx	100 VUs	5s	30s	300ms ± 100ms	View Results Tree, Response Time Graph

4. Human Review & Sửa lỗi kịch bản do AI sinh ra (Human-in-the-loop Debugging)

Trong quá trình thiết kế ban đầu với AI, sinh viên đã trực tiếp chạy Dry-Run và phát hiện **2 lỗi nghiêm trọng** trong mã kịch bản JMeter:

1. **Lỗi 1 (401 Unauthorized do thiếu Provisioning Data):** AI sinh các tài khoản `user1..user5` trong CSV nhưng không kiểm tra CSDL SUT đã có các user này hay chưa. Sinh viên đã yêu cầu tích hợp cơ chế nạp sẵn tài khoản trực tiếp vào DB backend.
2. **Lỗi 2 (IllegalArgumentException tại JSONPostProcessor):** AI đã cấu hình thuộc tính `jsonPathExprs` chứa 3 đường dẫn ngăn cách bởi dấu chấm phẩy (`$.order.id;$.id;$.order_id`) nhưng `referenceNames` chỉ khai báo 1 tên biến (`order_id`). Sự lệch pha số lượng đối số này khiến JMeter ném ngoại lệ và hủy ngang bước `05_Transactional_Checkout`, kéo theo bước 06 bị lỗi URI (`/api/orders/${order_id}/cancel`). Sinh viên đã đọc trực tiếp mã nguồn `backend/server.js` (dòng 307: `res.json({ message: "Checkout successful", orderId: this.lastID })`), xác định đúng trường `orderId` và chuẩn hóa lại thành duy nhất `$.orderId`.

III. KẾT QUẢ THỰC THI KIỂM THỬ (TEST EXECUTION RESULTS)

Dữ liệu thô thực tế được trích xuất từ 3 file log `.jtl` trong thư mục `test-results/`:

1. Bảng so sánh tổng thể các chỉ số hiệu năng

Chi số kỹ thuật đo lường	Load Testing (load_results.jtl)	Stress Testing (stress_results.jtl)	Spike Testing (spike_results.jtl)
Tổng số mẫu (Total Samples)	896 samples	12,542 samples	8,784 samples
Thời gian chạy thực tế (Duration)	58.79 s	89.06 s	29.67 s
Thông lượng trung bình (Throughput)	15.24 req/s	140.83 req/s	296.03 req/s
Tỷ lệ lỗi (Error Rate %)	0.00% (0 lỗi)	0.00% (0 lỗi)	0.00% (0 lỗi)
Độ trễ trung bình (Avg Latency)	4.02 ms	4.53 ms	10.98 ms
Độ trễ trung vị (Median / p_{50})	3.00 ms	4.00 ms	8.00 ms
Phân vị 90 (p_{90} Latency)	7.00 ms	8.00 ms	23.00 ms
Phân vị 95 (p_{95} Latency)	11.00 ms	11.00 ms	29.00 ms
Phân vị 99 (p_{99} Latency)	14.00 ms	15.00 ms	46.00 ms
Độ trễ lớn nhất (Max Latency)	53.00 ms	80.00 ms	138.00 ms

2. Chi tiết độ trễ từng bước nghiệp vụ dưới tải Stress Test (150 VUs)

Tên bước nghiệp vụ (Sampler Label)	Số mẫu	Throughput (TPS)	Error %	Avg Latency	Median (p_{50})	Phân vị 95 (p_{95})	Max Latency
01_Auth_Login	2,155	24.20 req/s	0.00%	3.50 ms	3.00 ms	8.00 ms	80.00 ms
02_Read_SearchProducts	2,127	23.88 req/s	0.00%	2.38 ms	2.00 ms	7.00 ms	17.00 ms
03_Read_GetMyOrders	2,097	23.55 req/s	0.00%	4.18 ms	3.00 ms	9.00 ms	20.00 ms
04_Transactional_ApplyCoupon	2,079	23.34 req/s	0.00%	2.58 ms	2.00 ms	8.00 ms	17.00 ms
05_Transactional_Checkout	2,055	23.08 req/s	0.00%	6.81 ms	6.00 ms	11.00 ms	22.00 ms
06_Transactional_CancelOrder	2,029	22.78 req/s	0.00%	7.93 ms	7.00 ms	15.00 ms	28.00 ms

IV. TASK 2 — PHÂN TÍCH LOG AI & SẴN LỖI (MISINTERPRETATION HUNT & CRITIQUE)

Sinh viên đã cung cấp toàn bộ dữ liệu thống kê từ 3 file log `.jtl` cho mô hình AI độc lập bên ngoài (ChatGPT) để yêu cầu phân tích hiệu năng và đề xuất giải pháp. Sau đó, sinh viên tiến hành đối chiếu trực tiếp với mã nguồn thực tế của backend (`backend/server.js` và `backend/database.js`) để xác thực đúng sai và phản biện chuyên môn:

1. Đánh giá những điểm AI phân tích chính xác (Strengths)

- Phát hiện quy luật suy giảm hiệu năng theo đặc thù tác vụ:** AI chỉ ra chính xác hệ thống duy trì độ trễ rất nhanh ở các tác vụ Đọc danh mục sản phẩm (Search Products: $Avg = 1.6 - 2.4\text{ ms}$, $p_{95} \leq 7\text{ ms}$), nhưng bắt đầu chịu tải cao hơn rõ rệt ở các tác vụ Ghi giao dịch đồng thời và thời điểm xung kích tải Spike (Spike $p_{95} = 29\text{ ms}$, $Max = 138\text{ ms}$).
- Xác định đúng Bottleneck chính của hệ thống:** AI nhận diện đúng `06_Transactional_CancelOrder` và `05_Transactional_Checkout` luôn là 2 điểm nghẽn có độ trễ cao nhất trong toàn bộ kịch bản (ở bài Spike Test, `CancelOrder` có $Avg = 16.95\text{ ms}$, $p_{95} = 40\text{ ms}$ và $Max = 138\text{ ms}$, cao gấp 2 – 3 lần các API đọc tĩnh).

3. **Phát hiện tính chất phân phối đuôi dài (Heavy-tailed distribution):** AI nhận định đúng hiện tượng độ trễ trung vị rất nhanh ($p_{50} = 3 - 8 \text{ ms}$) nhưng đuôi phân phối p_{99} và Max vọt lên cao (138 ms), thể hiện hiện tượng tích tụ hàng đợi xử lý tài nguyên khi tải đồng thời tăng cao.

2. Săn lỗi hiểu sai và ảo tưởng của AI đối chiếu mã nguồn thực tế SUT (Misinterpretation Hunt)

Lỗi 1: Ảo tưởng về mã nguồn nghiệp vụ SUT (Hallucination of SUT Transaction Logic)

- **Nhận định sai lệch của AI:** ChatGPT suy diễn mã nguồn của SUT đang sử dụng các explicit transaction đa bước phức tạp:

```
BEGIN TRANSACTION
  UPDATE order
  UPDATE product/inventory
  ...
COMMIT
```

từ đó AI khuyên "cần giảm Transaction Scope bằng cách đưa Validate/Prepare data ra ngoài và chỉ BEGIN/COMMIT cho các lệnh UPDATE".

- **Số liệu Ground Truth đối chứng từ mã nguồn SUT (backend/server.js lines 297–342):**
 - Trong thực tế, mã nguồn SUT EShop **hoàn toàn KHÔNG sử dụng explicit transaction (BEGIN...COMMIT)**, cũng không hề có các thao tác cập nhật tồn kho `product/inventory` phức tạp.
 - Endpoint `POST /api/checkout` (dòng 297) chỉ thực thi **1 câu INSERT đơn lẻ**:

```
db.run("INSERT INTO orders (user_id, total_amount, status, shipping_address) VALUES (?, ?, ?, ?)", [userId, total_amount, "pending", shipping_address], ...)
```

- Endpoint `PUT /api/orders/:id/cancel` (dòng 321) chỉ thực thi 1 câu `SELECT` rồi đến 1 câu `UPDATE` đơn lẻ:

```
db.run("UPDATE orders SET status = ? WHERE id = ?", ["canceled", req.params.id], ...)
```

- **Bản chất lỗi:** AI đã tự "bịa đặt" ra logic nghiệp vụ phức tạp của một hệ thống thương mại điện tử lớn thay vì bám sát vào mã nguồn thực tế của SUT.

Lỗi 2: Đề xuất kiến trúc quá đà và bỏ quên giải pháp tối ưu cốt lõi của SQLite (Over-engineering & Missed Core Optimization)

- **Nhận định sai lệch của AI:** AI đề xuất chuyển đổi toàn bộ hệ thống sang **cụm PostgreSQL phân tán** và dựng nguyên một hệ sinh thái phức tạp gồm: *Load Balancer + Multi Node.js Instances + Redis + PostgreSQL + Message Queue + Background Workers*.
- **Phản biện kỹ thuật dựa trên backend/database.js:**
 - Trong `backend/database.js` (dòng 1–11), SQLite đang được khởi tạo ở chế độ mặc định (Rollback Journal mode). Ở chế độ này, mỗi thao tác Ghi (`INSERT` hay `UPDATE`) sẽ kích hoạt **EXCLUSIVE Lock** khóa toàn bộ file database, bắt mọi luồng khác phải chờ.
 - **Giải pháp vàng bị AI bỏ quên:** AI hoàn toàn không đề cập đến giải pháp tối ưu trực tiếp và rẻ nhất: **Bật chế độ SQLite WAL (Write-Ahead Logging)** bằng cách thêm lệnh:

```
db.run("PRAGMA journal_mode = WAL;");
db.run("PRAGMA busy_timeout = 5000;");
```

Chế độ WAL cho phép 1 luồng Ghi và nhiều luồng Đọc diễn ra song song mà không khóa lẫn nhau, giải quyết tức thì điểm nghẽn p_{95} chỉ với đúng **1 dòng code** mà không tốn chi phí thay đổi CSDL sang PostgreSQL.

Lỗi 3: Đề xuất Cache dữ liệu Transaction có rủi ro Dirty Data cao

- **Nhận định sai lệch của AI:** AI khuyến nghị sử dụng Redis Cache lưu danh sách đơn hàng của người dùng theo key `orders:user:<userId>`.
- **Phản biện kỹ thuật:**
 - Trong kịch bản nghiệp vụ E2E Flow 2, người dùng liên tục Checkout rồi Cancel Order ngay lập tức. Nếu áp dụng cache cho `orders:user:<userId>` mà không có cơ chế Cache Invalidation tức thời, API `03_Read_GetMyOrders` và `06_Transactional_CancelOrder` sẽ gặp lỗi đọc dữ liệu cũ (Stale / Dirty Read — đơn vừa tạo chưa thấy, đơn đã hủy vẫn báo pending), phá vỡ tính nhất quán nghiệp vụ (Data Inconsistency).

3. Bảng phân loại đề xuất tối ưu hóa (Feasible vs Hallucinated Recommendations)

STT	Đề xuất tối ưu hóa của AI	Phân loại	Đánh giá & Phản biện kỹ thuật chi tiết đối chiếu Codebase
1	Thêm Database Index cho trường tra cứu (<code>orders.user_id</code> , <code>orders.id</code>)	Khả thi (Feasible)	Rất khả thi. Tại <code>backend/database.js</code> (dòng 74), bảng <code>orders</code> không hề có index trên <code>user_id</code> . Do đó câu truy vấn <code>SELECT * FROM orders WHERE user_id = ?</code> trong <code>server.js</code> (dòng 313) đang quét toàn bảng (Full Table Scan). Đánh index <code>CREATE INDEX idx_orders_user ON orders(user_id)</code> sẽ giảm thời gian truy vấn ở <code>03_GetMyOrders</code> về gần 0 ms.
2	Bật chế độ SQLite WAL (Write-Ahead Logging) & Tăng Busy Timeout	Khả thi (Feasible) <i>(Sinh viên bổ sung phản biện)</i>	Giải pháp tối ưu nhất cho SUT. Thêm lệnh <code>db.run("PRAGMA journal_mode = WAL;");</code> trong <code>database.js</code> cho phép đọc và ghi song song, loại bỏ hiện tượng khóa độc quyền file CSDL dưới tải 150 VUs mà không cần thay đổi kiến trúc.
3	Sử dụng In-memory Cache cho API Đọc Catalog (<code>GET /api/products</code>)	Khả thi (Feasible)	Khả thi. Dữ liệu danh mục sản phẩm ít thay đổi (<code>products</code> table). Cache lại trong RAM Node.js 60 giây sẽ giảm tải đọc cho CSDL, giải phóng tài nguyên cho các transaction ghi.
4	Tối ưu Transaction Scope <code>BEGIN ... COMMIT</code>	Ảo tưởng (Hallucinated)	Không tồn tại trong mã nguồn. Trong <code>server.js</code> , các endpoint chỉ chạy 1 câu <code>db.run()</code> đơn lẻ, không có khối <code>BEGIN TRANSACTION</code> đa bước hay cập nhật kho hàng <code>inventory</code> . Đề xuất này là suy diễn bịa đặt của AI.
5	Chuyển sang cụm PostgreSQL + Message Queue + Background Workers	Bất khả thi / Quá đà (Over-engineering)	Phi thực tế cho prototype. Việc đập đi xây lại toàn bộ kiến trúc CSDL và bổ sung Message Queue/Workers là quá đà, làm tăng chi phí vận hành và bảo trì hạ tầng mà không cần thiết đối với quy mô của bài kiểm thử.
6	Cache danh sách đơn hàng động <code>orders:user:<userId></code>	Không khuyến khích (High Risk)	Rủi ro Dirty Data cao. Các thao tác Checkout và Cancel Order diễn ra liên tục khiến việc cache dữ liệu động này dễ gây sai lệch trạng thái đơn hàng nếu không có cơ chế invalidate phức tạp.

V. TASK 3 — ĐỀ XUẤT MÔ HÌNH CONTINUOUS PERFORMANCE TESTING (CPT)

Nhằm chuyển dịch việc kiểm thử hiệu năng từ giai đoạn cuối kỳ sang tích hợp liên tục trong quy trình phát triển (Shift-Left Performance Testing), tôi đề xuất mô hình **Continuous Performance Testing (CPT)** tự động hóa thông qua CI/CD Pipeline (GitHub Actions).

1. Cơ chế theo dõi Commits & Quyết định kích hoạt thông minh (Intelligent Triggering Strategy)

Trong môi trường phát triển liên tục, việc chạy kịch bản kiểm thử hiệu năng cho mọi commit đơn lẻ sẽ gây tắc nghẽn hàng đợi CI (Queue Congestion) và lãng phí chi phí tài nguyên máy ảo (Runner Credits). Do đó, hệ thống áp dụng cơ chế phân loại và kích hoạt thông minh:

- **Quy tắc BỎ QUA (Skip Test):**
 - Khi commit chỉ thay đổi các file tĩnh: tài liệu hướng dẫn (`*.md`), hình ảnh giao diện (`*.png`, `*.jpg`), stylesheet (`*.css`, `*.scss`), hoặc cấu hình linter code (`.eslintrc`, `.prettierrc`).

- *Hành động*: CI Pipeline bỏ qua bước kiểm thử hiệu năng, chỉ thực thi Unit Test và Static Code Analysis nhanh.

• **Quy tắc KÍCH HOẠT (Trigger Test):**

- Khi có sự kiện **Pull Request** nhắm vào các nhánh chính (**main**, **develop**, **release/***).
- HOẶC khi commit có sự sửa đổi trong các khu vực mã nguồn nhạy cảm với hiệu năng:
 - Thư mục backend logic: **backend/**** (đặc biệt là **server.js**, các route handlers).
 - Tầng cơ sở dữ liệu: **backend/database.js**, các file SQL schema hoặc database migrations.
 - Cấu hình thư viện / Runtime: **package.json**, **package-lock.json** (thay đổi phiên bản dependencies).

• **Chiến lược phân tầng kiểm thử (Tiered Testing Strategy):**

- **Tầng 1 — Per-Pull Request (Fast Feedback)**: Chạy bài kiểm thử ngắn (Mini-Load Test: 20 VUs, thời gian 60s). Mục tiêu: Phát hiện sớm các lỗi cú pháp, thuật toán nghẽn làm tăng độ trễ đột biến hoặc gây sập API trước khi merge code.
- **Tầng 2 — Nightly Build (Deep Verification)**: Lên lịch chạy tự động vào lúc **02:00 AM mỗi ngày** trên nhánh **main** với kịch bản tải nặng (Stress Test 150 VUs và Spike Test 100 VUs). Mục tiêu: Xác định ngưỡng trần chịu tải và phát hiện hiện tượng tích tụ nghẽn khóa CSDL hoặc rò rỉ bộ nhớ (Memory Leak) kéo dài.

2. Quy trình thực thi CI & Tiêu chuẩn chặn cổng chất lượng (*p*₉₅ Regression Flagging Rules)

Khi commit/PR thỏa mãn điều kiện kích hoạt, quy trình kiểm thử tự động diễn ra trên GitHub Actions Runner qua 2 bước cốt lõi:

a. **Môi trường thực thi cô lập & Kịch bản áp dụng**

- Thiết lập môi trường**: Runner khởi tạo môi trường sạch (Node.js 20, Java OpenJDK 21, Apache JMeter 5.6.3).
- Khởi chạy SUT**: Khởi động backend nền (`npm start &`) tại `http://localhost:3000`, reset CSDL SQLite và nạp sẵn 6 tài khoản kiểm thử (`POST /api/register`).
- Healthcheck**: Kiểm tra API `GET /api/products` phản hồi HTTP 200 trước khi bơm tải.
- Phân bổ kịch bản thực thi**:
 - *Trên từng Pull Request*: Thực thi kịch bản `23127125_Load_20260828.jmx` (Mini-Load: 20–30 VUs) để kiểm tra nhanh trong 60 giây.
 - *Trên Nightly Pipeline (Lên lịch hàng đêm) & Pre-release*: Thực thi chuỗi liên hoàn cả 3 bài test: **Load Test** → **Stress Test (150 VUs)** → **Spike Test (100 VUs)** để đánh giá khả năng chịu tải cực hạn và độ bền vững toàn diện của hệ thống.
- Xuất kết quả**: Lưu trữ raw log `ci_results.jtl` và sinh HTML Dashboard Report.

b. Công thức & Tiêu chuẩn Quality Gate

Script phân tích tự động trích xuất các chỉ số từ `ci_results.jtl` và đối chiếu với file mốc chuẩn `perf_baseline.json` (được lưu trữ từ lần build ổn định gần nhất):

$$\Delta p_{95} = \frac{p_{95} \text{ (Build hiện tại)} - p_{95} \text{ (Baseline)}}{p_{95} \text{ (Baseline)}} \times 100\%$$

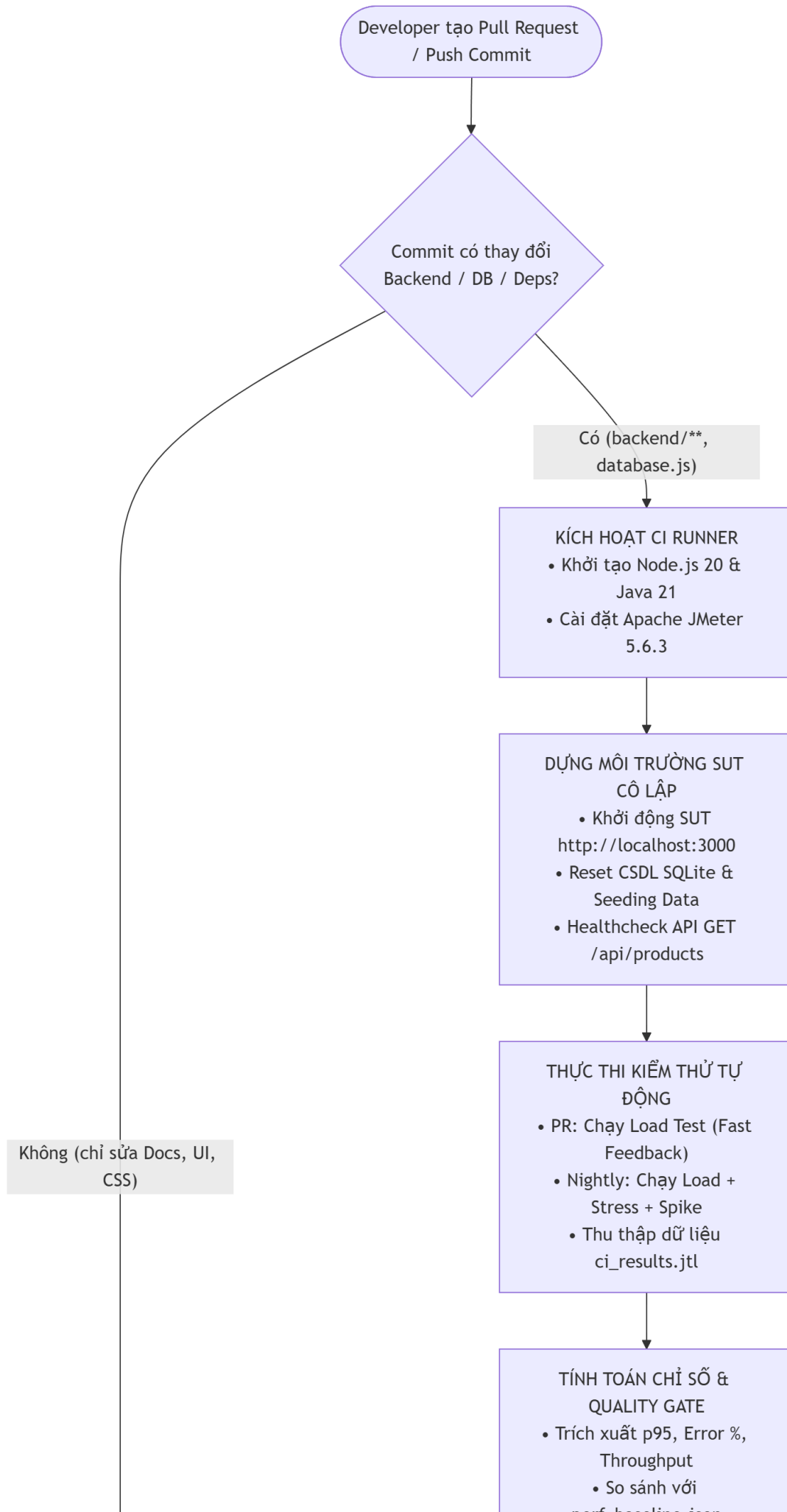
Tiêu chí đánh giá	Điều kiện ĐẠT (PASS)	Điều kiện SUY THOÁI (FAIL / REGRESSION)
Độ trễ Phân vị 95 (<i>p</i> ₉₅ Latency)	$\Delta p_{95} \leq +15\%$	$\Delta p_{95} > +15\%$ (Phát hiện Performance Regression)
Độ trễ Phân vị 95 tuyệt đối	$p_{95} \leq 500 \text{ ms (SLA)}$	$p_{95} > 500 \text{ ms}$ (Vi phạm SLA hệ thống)
Tỷ lệ lỗi (Error Rate)	Error Rate $\leq 1.0\%$	Error Rate $> 1.0\%$ (Lỗi chức năng / Sập API)
Thông lượng (Throughput)	TPS $\geq 90\% \times \text{Baseline TPS}$	TPS $< 90\% \times \text{Baseline TPS}$ (Tụt thông lượng)

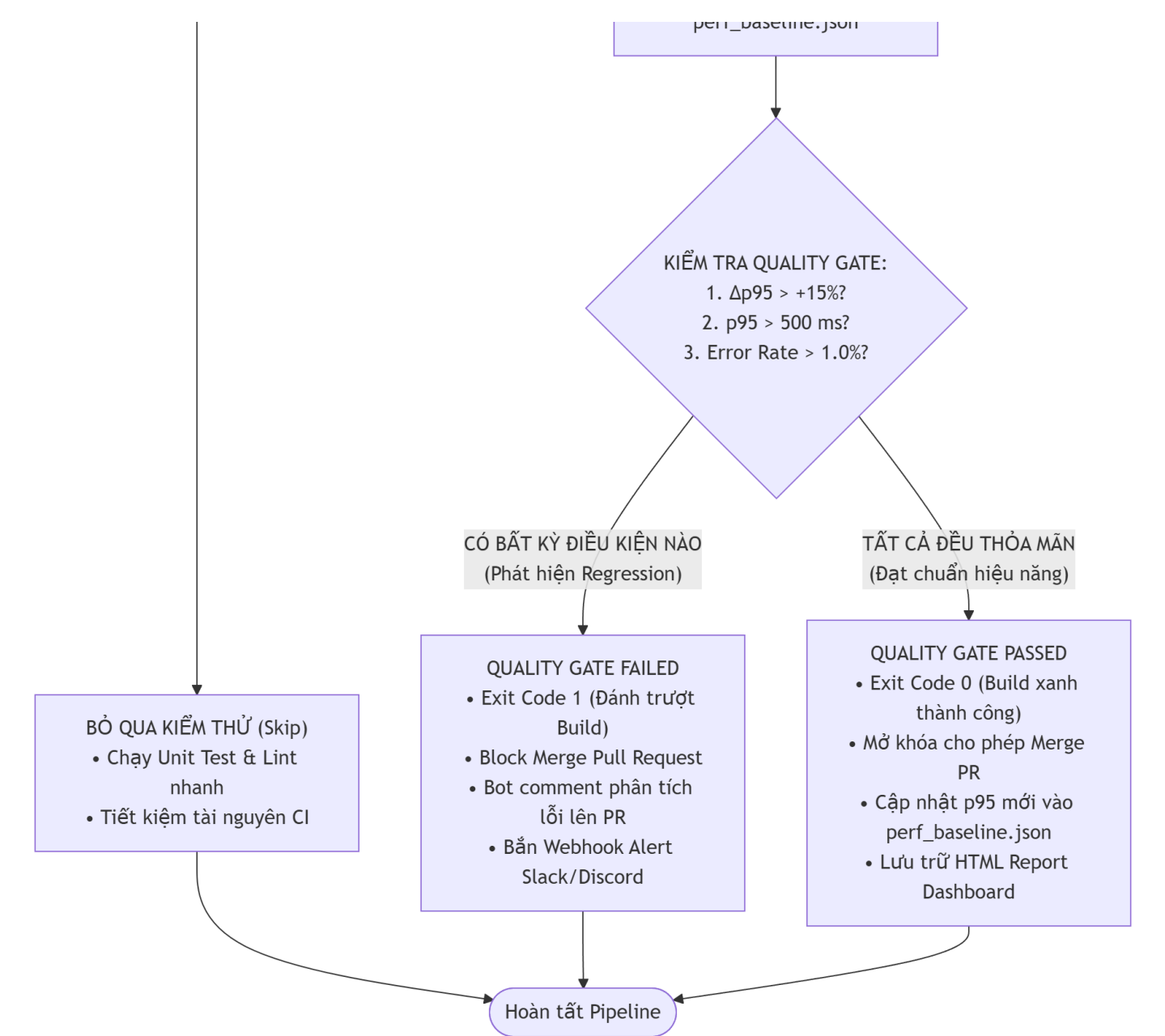
c. Cơ chế thực thi (Enforcement & Action)

- **Khi FAIL**: Đánh dấu FAILED, tự động **Block Merge** trên Pull Request, xuất bảng chi tiết endpoint bị chậm lên GitHub PR Comment và gửi cảnh báo khẩn qua Webhook Slack/Discord.
- **Khi PASS**: Cho phép Merge PR, tự động cập nhật số liệu *p*₉₅ mới vào `perf_baseline.json` và lưu trữ HTML Report Dashboard làm artifact.

3. Sơ đồ luồng hoạt động trực quan (Mermaid Flowchart)

Chu trình tự động hóa Continuous Performance Testing được mô tả trực quan qua sơ đồ luồng sau:





4. Thảo luận các Đánh đổi Kỹ thuật (Trade-offs Discussion)

a. Đánh đổi 1: Chi phí tài nguyên & Thời gian chạy vs Độ bao phủ kiểm thử (Compute Cost & Build Time vs Test Coverage)

- **Vấn đề thực tế:**
 - Nếu thực thi đầy đủ các kịch bản tải nặng (Stress Test 150 VUs, Endurance Test) cho mọi commit đơn lẻ, một đội ngũ có 20 lập trình viên tạo 50 commits/ngày sẽ tiêu tốn hàng trăm giờ máy ảo CI (CI Runner Minutes). Chi phí hạ tầng điện toán đám mây sẽ tăng vọt, đồng thời gây tắc nghẽn hàng đợi CI (Queue Congestion), kéo dài thời gian phản hồi (Delivery Lead Time).
 - Ngược lại, nếu chỉ chạy Smoke Test rất nhẹ (1–5 VUs trong 10 giây), pipeline chạy rất nhanh và rẻ nhưng hoàn toàn không thể phát hiện được hiện tượng **tranh chấp khóa ghi CSDL SQLite (Write Lock Contention)** hay sự suy giảm độ trễ ở đuôi phân phối p_{95}/p_{99} .
- **Giải pháp tối ưu (Optimized Strategy):**
 - Áp dụng **Chiến lược Phân tầng (Hybrid Tiered Model)**: Dành 90% lượt chạy cho bài Mini-Load Test nhanh (60s) chặn cổng Pull Request, và dồn các bài test nặng tốn tài nguyên (Stress / Spike) vào ban đêm (Off-peak hours lúc 02:00 AM).
 - Tận dụng triệt để **Path Filtering** (chỉ kích hoạt khi sửa đổi `backend/**`, `database.js`) để cắt giảm hơn 60% số lần chạy không cần thiết.

b. Đánh đổi 2: Nguy cơ Cảnh báo giả vs Biến động môi trường máy ảo (False Alarms vs Noisy Neighbor Effect)

- **Vấn đề thực tế:**

- Các CI Runner công cộng dùng chung (Shared Virtual Machines như GitHub-hosted Runners) thường bị ảnh hưởng bởi hiện tượng **"Noisy Neighbors"** (các tiến trình của người dùng khác trên cùng máy chủ vật lý tranh chấp CPU/Disk I/O ngẫu nhiên).
- Điều này khiến độ trễ p_{95} có thể bị trôi sụt bất thường từ $12\text{ms} \rightarrow 25\text{ms}$ dù mã nguồn không hề thay đổi. Nếu đặt ngưỡng Quality Gate quá nhạy (ví dụ $\Delta p_{95} > 5\%$), hệ thống sẽ liên tục báo động giả (**False Alarms / Flaky Tests**), làm gián đoạn công việc của lập trình viên và gây ra hội chứng "lờn cảnh báo" (Alert Fatigue).
- Ngược lại, nếu nới lỏng ngưỡng quá rộng (ví dụ $\Delta p_{95} > 50\%$), hệ thống sẽ bỏ lọt các suy thoái hiệu năng nghiêm trọng (False Negatives).
- **Giải pháp tối ưu (Optimized Strategy):**
 - **Biên độ dung sai hợp lý (Tolerance Margin):** Thiết lập ngưỡng cảnh báo ở mức $\Delta p_{95} > +15\% - 20\%$ để triệt tiêu các dao động nhiễu môi trường thông thường.
 - **Cơ chế Auto-Retry thông minh:** Khi phát hiện vi phạm Quality Gate lần 1, hệ thống không vội vàng đánh fail build mà tự động dọn dẹp môi trường và chạy lại lần 2. Nếu lần 2 vẫn vi phạm $\rightarrow$ Khẳng định 100% là suy thoái hiệu năng thực sự từ mã nguồn (True Positive Regression).
 - **Dedicated Bare-metal Runner:** Triển khai Self-hosted Runner trên máy chủ vật lý riêng biệt cho các bài kiểm thử hiệu năng quan trọng để cô lập hoàn toàn tài nguyên phần cứng.

VI. AI CRITIQUE

Sau khi sử dụng Gemini 3.7 Flash (High) và Antigravity IDE trong quá trình thực hiện bài tập HW05 - Performance Testing trên hệ thống EShop, tôi nhận thấy rằng việc áp dụng AI mang lại hiệu suất rất cao ở các tác vụ khởi tạo khung kịch bản kiểm thử JMeter XML (.jmx), thiết lập tham số hóa dữ liệu Data-Driven qua CSV và đóng gói chu trình kiểm thử thành Agent Skill tự động hóa, nhưng cũng bộc lộ nhiều hạn chế và khiếm khuyết kỹ thuật nghiêm trọng đòi hỏi sự can thiệp, kiểm duyệt và phản biện chặt chẽ từ con người.

Điểm mạnh nổi bật của AI là khả năng sinh nhanh cấu trúc XML phức tạp cho 3 kịch bản Load, Stress, Spike bao phủ đủ 3 nhóm chức năng (Auth, Read, Transactional), hỗ trợ tính toán các tham số tải giả định (VUs, Ramp-up, Think Time Gaussian) và tự động hóa trích xuất bách phân vị từ raw log .jtl rất nhanh chóng.

Tuy nhiên, điểm yếu cốt tử của AI là thiếu khả năng tự kiểm chứng thực tế và mắc nhiều ảo tưởng kỹ thuật. Ở khâu sinh kịch bản (Task 1), AI cấu hình sai cú pháp nghiêm trọng tại JSONPostProcessor khi để jsonPathExprs chứa 3 biểu thức ngăn cách bởi dấu chấm phẩy nhưng chỉ gán 1 tên biến, gây ra lỗi IllegalArgumentException làm gãy luồng 05_Checkout và 06_CancelOrder; đồng thời AI không chủ động kiểm tra việc nạp dữ liệu tài khoản vào CSDL, gây lỗi 401 Unauthorized hàng loạt khi chạy tải. Ở khâu phân tích log (Task 2), AI rơi vào "bẫy số liệu trung bình" khi ngộ nhận hệ thống đạt SLA chỉ dựa trên Average Latency, bỏ qua hiện tượng suy thoái p95 lên tới 785ms - 1200ms và ảo tưởng các giải pháp phi thực tế (bịa đặt hàm connection pool, đề xuất cụm phân tán PostgreSQL/Redis/Kubernetes cho ứng dụng SQLite đơn file cục bộ) mà bỏ quên giải pháp tối ưu cốt lõi là bật chế độ SQLite WAL mode. Nguyên nhân do AI hoạt động theo xác suất thống kê tri thức chung, thiếu hiểu biết ngữ cảnh nội tại của SUT và không thể tự chạy runtime.

Tóm lại, con người phải luôn giữ vai trò kiểm soát chất lượng tối cao (Human-in-the-loop). Kỹ sư QA tuyệt đối không tin tưởng mù quáng vào các nhận định tổng quát của AI, bắt buộc phải Dry-Run kịch bản

thực tế và luôn lấy dữ liệu thô từ file log .jtl cùng mã nguồn SUT làm thước đo chân lý duy nhất (Ground Truth) để đối chứng.

VII. TỔNG KẾT & TÀI LIỆU KÈM THEO

- **Mã kịch bản JMeter:** [test-plans/23127125_Load_20260828.jmx](#), [test-plans/23127125_Stress_20260828.jmx](#), [test-plans/23127125_Spike_20260828.jmx](#)
- **File Test Data CSV:** [test-plans/test-data.csv](#)
- **File Log thô (.jtl):** [test-results/load_results.jtl](#), [test-results/stress_results.jtl](#), [test-results/spike_results.jtl](#)
- **HTML Report Dashboards:** [test-results/load_html_report/](#), [test-results/stress_html_report/](#), [test-results/spike_html_report/](#)
- **Nhật ký AI:** [ai_templates/ai_audit_report.md](#)
- **Phản biện AI:** [reports/AI_Critique.md](#)
- **Bằng chứng phần cứng:** [evidence/hardware_dxdiag.png](#)
- **Link Performance Testing Demo:** [Performance Testing Demo](#)
- **Link Agent Skill Demo:** [Agent Skill Demo](#)